

For production builds, it always compiles in release mode and strips debug symbols:

```
cargo build --release
strip target/release/ultra_api
```

You can optimise it more in *Cargo.toml*:

```
opt-level = "z"
lto = true
codegen-units = 1
panic = "abort"
```

Finally, structure projects for scalability. Instead of adding everything into *main.rs*, split logic into modules:

```
src/
├─ main.rs
├─ routes/
│   └─ mod.rs
├─ handlers/
│   └─ mod.rs
└─ services/
    └─ mod.rs
```

Initialise modules like this:

```
mod routes;
mod handlers;
mod services;
```

This modular workspace keeps your API manageable as it grows. For each layer, whether it's routes, handlers or services, scaling features is easy and doesn't lead to chaos.

Core pillars of Actix Web

Actix Web's performance is not by accident, but by design and is built on a few powerful core pillars. Once you know these, writing high-performance APIs becomes much more intuitive. First up, the async runtime. Actix Web is built on top of Tokio that allows for non-blocking I/O. You just mark handlers as async and Actix does the rest:

```
use actix_web::{get, Responder};
#[get("/async")]
async fn async_handler() -> impl Responder {
    "Handled without blocking!"
}
```

Next, we have extractors, one of the most elegant features of Actix. They allow to pull structured data out of requests in a safe way — no manual parsing.

```
use actix_web::{get, web, Responder};
#[get("/user/{id}")]
async fn get_user(path: web::Path<u32>) -> impl Responder
{
    format!("User ID: {}", path.0)
}
```

You can extract JSON payloads:

```
use actix_web::{post, web, Responder};
use serde::Deserialize;
#[derive(Deserialize)]
struct User {
    name: String,
}
#[post("/create")]
async fn create_user(user: web::Json<User>) -> impl Responder
{
    format!("Created user: {}", user.name)
}
```

Actix provides *web::Data<T>* for state sharing. This is a safe way to share application state between threads. Internally it uses Arc, so cloning is cheap and thread safe:

```
use actix_web::{web, App};
struct AppState {
    counter: i32,
}
let state = web::Data::new(AppState { counter: 0 });
App::new()
    .app_data(state.clone());
```

Finally, understand the request lifecycle. *HttpServer* spins up workers, *App* configures routes and middleware, and *service* handles requests:

```
HttpServer::new(|| {
    App::new()
        .service(get_user)
        .service(create_user)
})
    .bind(("127.0.0.1", 8080))?
    .run()
    .await;
```

Asynchronous runtime: This runs async tasks on Tokio, so it can process requests at the same time and not block threads. This leads to more throughput and low latency even under heavy load.

Extractors: Extractors automatically extract request data into strongly typed Rust structures. This offers compile-time